

Group 6: Background

Jun Tang
Fairleigh Dickinson University
Vancouver, Canada
j.tang4@student.fdu.edu

Xiaoyi Han
Fairleigh Dickinson University
Vancouver, Canada
x.han1@student.fdu.edu

Kaiwen Wang
Fairleigh Dickinson University
Vancouver, Canada
k.wang4@student.fdu.edu

Ziyan Qiu
Fairleigh Dickinson University
Vancouver, Canada
z.qiu1@student.fdu.edu

Hongyi Niu
Fairleigh Dickinson University
Vancouver, Canada
h.niu@student.fdu.edu

1 Background

1.1 Flash Cache and Disk-head Time (DT)

The speed of data growth is exponential [4]. Storage systems based on a hard disk drive (HDD) eventually fail to meet high-throughput requirements [3]. According to [2], on average, flash drives deliver 15–20 times the IOPS of hard disk drives and provide throughput 700–1000 times higher. Consequently, a modern storage stack is typically a layered system that serves client I/O requests through a hierarchy of devices. For cost considerations, it is still backed by HDD bulk storage, but to mitigate high I/O demand, a flash cache layer is placed in front to reduce peak backend load. The flash cache stages data in units of fixed-size segments, which allows it to spread out the cost of writing to flash while aligning with HDD read sizes for efficiency. The time for one HDD access consists of two parts: seek and move the head, and rotate the platter to read and transfer. The authors define Disk-head Time (DT) Eq. (1) where t_{seek} is the average seek time and t_{read} is the per-byte transfer time [5].

$$DT_i = t_{\text{seek}} + n \cdot t_{\text{read}} \quad (1)$$

This means HDDs have two constraints upon their throughput: IOPS (input/output operations per second) and sequential transfer bandwidth. Small I/Os are limited by IOPS, and large I/Os will hit the bottleneck of disk bandwidth.

The authors emphasize DT “balances IOPS and bandwidth” and “generalizes across variable-size I/Os” [5]. Regardless of request size, using DT as the unified optimization objective allows cache admission and prefetching policies to effectively reduce peak backend load on HDDs. (Word count: 252)

1.2 Episode Model and OPT (Oracle)

The episodes model groups related cache accesses together instead of tracking every single one individually, which makes flash caching analysis way more manageable. An episode is basically a bunch of accesses to the same data block that would all be hits if we decide to cache that block initially. The first access is always going to be a miss since the data isn’t there yet, but everything else would be hits that can be efficiently served from the cache without additional disk activity.

They figure out episode boundaries using LRU eviction age. If two accesses happen within that time window, they are considered part of the same episode. If the gap is longer than eviction age, that’s where one episode ends and maybe a new one starts. This approach has nice advantages. Each episode can be analyzed independently, which makes the math way simpler. It also captures the main trade-off: you pay write cost upfront when caching data, then get benefits

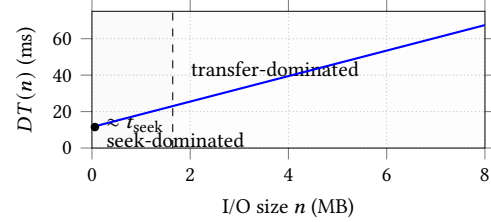


Figure 1: DT vs. I/O size ($t_{\text{seek}}=11.5$ ms, $BW=143$ MB/s). The line shows seek- and transfer-dominated regimes; crossover at $n^* \approx 1.64$ MB. Peak DT drives provisioning, which flash caching helps reduce.

from hits during that episode, essentially trading flash endurance for reduced disk load.

The OPT policy uses this to approximate optimal decisions. It generates episodes using assumed eviction age, scores each episode by Disk-head Time saved per storage cost, then picks highest-scoring ones until the write budget runs out. Scoring formula:

$$\text{Score}(\text{Episode}) = \frac{DT_{\text{Saved}}(\text{Episode})}{\text{Size}(\text{Episode})} \quad (2)$$

What’s interesting is how it changes ML training. Instead of learning from individual accesses with complicated dependencies, models focus on episode-level decisions which is much simpler. The episode boundaries represent key decision points where you choose whether to admit something or not, and this abstraction makes model design far more tractable in practice. (Word count: 250)

1.3 ML-Guided Admission (OPT Imitation for Peak DT)

Machine learning-guided admission in flash caches refers to the use of a predictive model to decide whether a missed block should be inserted into cache under a constrained flash write budget. The approach is based on imitating an offline reference policy, OPT, which provides a near-optimal bound for maximizing disk-head time (DT) savings.

The training of this policy relies on the episode model, which groups all accesses to a block that would be hits if the block were admitted at the first access. Each episode is assigned a cost (number of flash segments written) and a benefit (total DT saved). Episodes are scored by their benefit-to-cost ratio as in Eq. (2).

Episodes are ranked by this score, and the highest scores are selected until the flash write budget is met. These selected episodes provide the labels used to train the admission model. The model is implemented as a binary classifier, typically using gradient boosting

machines (GBMs). Input features include metadata describing the request context (namespace, user, block type) and dynamic features capturing recent access frequency. Samples from the first few accesses of each episode are included to capture early decision signals. The classifier outputs an admission probability, and a threshold is calibrated in simulation to achieve the desired flash write rate. The optimization target is Peak Disk-head Time, which reflects the highest backend load over a given interval. By focusing on DT rather than cache hit rate, the admission policy ensures that cache decisions reduce seek and transfer time, aligning with the objective of minimizing backend resource utilization. (Word count:258)

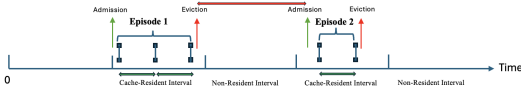


Figure 2: Episode Model Timeline. Two episodes of the same block are illustrated. Each episode begins with an Admission (green arrow), during which the block is Cache-Resident (green interval) and subsequent accesses are hits. If no access occurs for longer than the Eviction Age, the block is Evicted (red arrow), starting a Non-Resident Interval.

1.4 ML-Guided Prefetching (Coordinated with Admission)

ML-When is a classifier which aims to decide whether prefetching is worth executing for a given episode to achieve net reductions in DT [5]. Based on the admission result made by Baleen’s admission model, a binary classifier trained on episodes to imitate OPT [5]—ML-Range proposes a contiguous segment range. ML-When then judges this proposal by estimating the marginal DT benefit of prefetching. If the predicted benefit is greater than a threshold ($\epsilon \approx 5$ ms, about half a disk seek), the prefetch is carried out; otherwise, it is skipped. By doing this, Baleen proactively caches the high-value segments of the data that are most likely to reduce the peaks in DT. According to the paper, Baleen takes blocks (8 MB) as

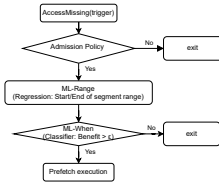


Figure 3: Runtime workflow of Baleen

the data boundary. Within each block, the unit of data is a segment (128 KB). When an episode is admitted, ML-When evaluates ML-Range’s predicted segment range. If ML-When affirms the decision, those segments are prefetched and written into flash cache.

ML-When evaluates ML-Range’s proposals using the benefit formulas (Eq. 3a–3e in [5]), which quantify the DT saved by prefetching versus not prefetching. If the predicted benefit exceeds a small threshold ($\epsilon \approx 5$ ms, about half a disk seek), ML-When approves the prefetch; otherwise, it is skipped. In this way, only high-value prefetches are admitted, avoiding unnecessary flash writes. (Word count: 253)

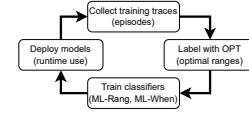


Figure 4: Training loop for Baleen. Runtime traces are labeled with OPT; ML-Range and ML-When are trained and deployed, producing new traces to continue the cycle.

1.5 Baleen-TCO (Cost-Aware Flash Write-Rate Control)

Total cost of ownership in Baleen was introduced as a consideration in cost controller mainly focused on balancing HDD cost and SSD endurance – more data request and transactions will occur on HDD directly if too little is written in SSD, raising cost due to affected performance costed by high peak DT and the demands for more HDD to maintain performance. Meanwhile, too much data written in the flash cache will accelerate abrasion on the SSD, which also increases cost.

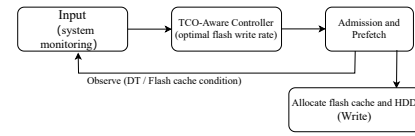


Figure 5: Control Diagram

In this article, to address this issue, Baleen continuously monitors DT metrics (such as average DT and peak DT) as well as flash memory health metrics (such as remaining write cycles). Flash write rate was calculated dynamically to optimize the cache effect without overuse consumption of flash memory. Baleen achieved a trade-off that minimizing the TCO by protecting flash cache from excessiveness while reducing DT to decrease hard disk load, instead of simply maximizing cache hits.

Admission policies must also consider flash write amplification and device endurance. For instance, CacheSack optimizes flash cache admission by trading off disk reads and flash writes in Google datacenters [6]. Recent studies show that sustainable cache designs need to manage backend load and also keep flash write amplification low to extend device life. For instance, WREN[1] shows how new flash interfaces can help reduce write loads without losing speed. This is similar to Baleen-TCO’s approach because both keep device costs in mind. (Word count:240)

1.6 Writing & Citation Checklist

The Background Section is written in cohesive prose, and each section (1.1–1.5) is connected well and makes sense to the next one.

In terms of citation, the background part has the main reference Baleen FAST ’24 paper and four more papers related to flash caching and machine learning-guided caching. The citations include the storage system architectures, data growth and storage problem, and coordinated flash cache management.

All citations are written in ACM style and no preprints are included. In terms of figures, all required figures are vector diagrams and each figure is accompanied by a descriptive caption. In conclusion, this checklist makes sure the writing, citation, and formatting requirements in the guidelines. (Word count:114)

Member Contributions

To start the paper, every team member read and understood the paper. Below are the contributions for each member:

Jun Tang (2111172)

- Read the paper and discussed with team members.
- Wrote subsection 1 *Flash Cache and Disk-head Time (DT)*
- Wrote subsection 4 *ML-Guided Prefetching (Coordinated with Admission)*.
- Drew necessary diagrams (training loop, DT vs. I/O Size).
- Team coordination - set up latex in Overleaf, kanban project plan.

Ziyan Qiu (2089937)

- Read the paper and discussed with team members.
- Wrote subsection 2 *Episode Model And OPT (Oracle)*.
- Inserted “Word count” lines and verified the two-page fit for the Background section.
- Reviewed the paper and provided solid comments for revising.
- Reviewed formatting and references against the rubric (ACM sigconf, vector graphics, sectioning) and coordinated fixes.

Kaiweng Wang (2073516)

- Read the paper and discussed with team members.
- Wrote subsection 3 *ML-Guided Admission (OPT Imitation for Peak DT)*.
- Drew the episode timeline diagram showing cache-resident/non-resident intervals with admission/eviction boundaries.
- Coordinated team logistics.

Xiaoyi Han (2133542)

- Wrote subsection 5 *Baleen-TCO*.
- Created diagrams showing TCO-aware controller.
- Summarized Baleen-TCO trade-offs for the team.
- Wrote clear TCO explanations.
- Verified accuracy against paper and related studies.

Hongyi Niu (2104640)

- Read the paper and discussed with team members.
- Reviewed all sections (1.1–1.5) to ensure cohesive prose, removing fragmented or list-style writing.
- Wrote subsection 6 *Writing & Citation Checklist*.
- Contributed to subsection 5 *Baleen-TCO*.
- Checked the team contribution and citation checklist.

References

- [1] George Amvrosiadis, Joseph McAllister, Ada Gavrilovska, and Gregory R. Ganger. 2024. WREN: A Sustainable Cache for Emerging Write-Read-Erase Flash Interfaces. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 1–17. <https://www.pdl.cmu.edu/PDL-FTP/NVM/2024-osdi-mcallister.pdf>
- [2] Jakob Lüttgau, Michael Kuhn, Kira Duwe, Yevhen Alforov, Eugen Betke, Julian Kunkel, and Thomas Ludwig. 2018. Survey of Storage Systems for High-Performance Computing. *Supercomputing Frontiers and Innovations* 5, 1 (2018), 31–58. doi:10.14529/jsfi180103
- [3] Zhiwei Ren, Lei Chen, Fan Wang, Weili Zhang, Min Wang, Jiale Ma, Rui Han, David K. Y. Chiu, and Xiaofeng Wang. 2019. Hybrid Storage Systems: A Survey of Architectures and Algorithms. *Comput. Surveys* 52, 4 (2019), 1–38. doi:10.1145/3323934
- [4] V. Turner, J. F. Gantz, D. Reinsel, and S. Minton. 2014. *The Digital Universe of Opportunities: Rich Data and the Increasing Value of the Internet of Things*. Technical Report. IDC Analysis Future. White Paper, IDC.
- [5] Daniel Lin-Kit Wong, Hao Wu, Carson Molder, Sathya Gunasekar, Jimmy Lu, Snehal Khandkar, Abhinav Sharma, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2024. Baleen: ML Admission & Prefetching for Flash Caches. In *Proceedings of the 22nd USENIX Conference on File and Storage Technologies (FAST 2024)*. USENIX Association, Santa Clara, CA, 347–371.
- [6] Tzu-Wei Yang, Seth Pollen, Mustafa Uysal, Arif Merchant, and Homer Wolfmeister. 2022. CacheSack: Admission Optimization for Google Datacenter Flash Caches. In *2022 USENIX Annual Technical Conference (USENIX ATC '22)*. USENIX Association, Carlsbad, CA, USA, 209–224. <https://www.usenix.org/conference/atc22/presentation/yang-tzu-wei>